4.1 Project Coding – Database, backend, and frontend

Seminar Hall Booking System – important code segments (schema, backend, frontend; full repo on GitHub)

Full source code – Included the most important code segments only (core schema, routing, JWT auth, request creation, department approval, overlap checks, and key UI logic). Omitted lines are marked with `// ...` or `-- ....` Full sources are on GitHub. Repository: https://github.com/sreejithr247/seminar-hall-booking-system.

Database schema (PostgreSQL)

Seminar Hall Booking System – full Tables.sql DDL

DB/Tables.sql

```
-- ============================================
-- SEMINAR HALL BOOKING SYSTEM SCHEMA
-- ============================================
-- 1. Custom ENUM Types
CREATE TYPE USER_ROLE AS ENUM ('admin', 'dept_coordinator', 'requester', 'faculty');
CREATE TYPE APPROVAL_STATUS AS ENUM ('pending', 'forwarded', 'approved', 'rejected', 'amended');
CREATE TYPE BOOKING_STATUS AS ENUM ('confirmed', 'cancelled', 'completed', 'no_show');
CREATE TYPE REQUESTER_TYPE AS ENUM ('class', 'club');
-- 2. Core Tables
CREATE TABLE departments (
    dept_id      SERIAL PRIMARY KEY,
    dept_name    VARCHAR(100) NOT NULL UNIQUE,
    dept_code    VARCHAR(10)  UNIQUE,
    created_at   TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE classes (
    class_id     SERIAL PRIMARY KEY,
    class_name   VARCHAR(100) NOT NULL,           -- e.g., BCA 2023-26
    dept_id      INTEGER NOT NULL REFERENCES departments(dept_id),
    year         VARCHAR(10),
    UNIQUE(dept_id, class_name)
);

CREATE TABLE clubs (
    club_id      SERIAL PRIMARY KEY,
    club_name    VARCHAR(150) NOT NULL UNIQUE,    -- e.g., Robotics Club
    dept_id      INTEGER REFERENCES departments(dept_id),  -- NULL = college-level
    description  TEXT
);

CREATE TABLE halls (
    hall_id      SERIAL PRIMARY KEY,
    hall_name    VARCHAR(100) NOT NULL,
    capacity     INTEGER NOT NULL CHECK (capacity > 0),
    location     VARCHAR(100),
    facilities   JSONB DEFAULT '{}',             -- {"projector": true, "ac": true}
    is_active    BOOLEAN DEFAULT true
);
-- 3. Users & Requesters (Class or Club representative)
CREATE TABLE users (
    user_id       SERIAL PRIMARY KEY,
    username      VARCHAR(50)  NOT NULL UNIQUE,
    password_hash VARCHAR(255) NOT NULL,
    full_name     VARCHAR(100) NOT NULL,
    email         VARCHAR(100) UNIQUE,
    phone         VARCHAR(15),
    role          USER_ROLE NOT NULL,
    dept_id       INTEGER REFERENCES departments(dept_id),
    is_active     BOOLEAN DEFAULT true,
    created_at    TIMESTAMPTZ DEFAULT NOW(),
    updated_at    TIMESTAMPTZ DEFAULT NOW()
);
-- One user can represent only ONE entity: either a Class or a Club
CREATE TABLE requesters (
    requester_id      SERIAL PRIMARY KEY,
    user_id           INTEGER NOT NULL UNIQUE REFERENCES users(user_id) ON DELETE CASCADE,
    requester_type    REQUESTER_TYPE NOT NULL,
    class_id          INTEGER REFERENCES classes(class_id),
    club_id           INTEGER REFERENCES clubs(club_id),
    created_at        TIMESTAMPTZ DEFAULT NOW(),
    CONSTRAINT valid_requester
        CHECK (
            (requester_type = 'class' AND class_id IS NOT NULL AND club_id IS NULL) OR
            (requester_type = 'club' AND club_id IS NOT NULL AND class_id IS NULL)
        )
);
-- 4. Booking Requests & Workflow
CREATE TABLE requests (
    request_id          SERIAL PRIMARY KEY,
    requester_id        INTEGER NOT NULL REFERENCES requesters(requester_id),
    hall_id             INTEGER NOT NULL REFERENCES halls(hall_id),
    event_title         VARCHAR(200) NOT NULL,
    event_description   TEXT,
    event_date          DATE NOT NULL,
    start_time          TIME NOT NULL,
```

```
    end_time            TIME NOT NULL,
    expected_attendees  INTEGER CHECK (expected_attendees > 0),
    purpose             VARCHAR(300),
    dept_status         APPROVAL_STATUS DEFAULT 'pending',
    dept_approved_by    INTEGER REFERENCES users(user_id),
    dept_approved_at    TIMESTAMPTZ,
    dept_remarks        TEXT,
    admin_status        APPROVAL_STATUS DEFAULT 'pending',
    admin_approved_by   INTEGER REFERENCES users(user_id),
    admin_approved_at   TIMESTAMPTZ,
    admin_remarks       TEXT,
    requested_at        TIMESTAMPTZ DEFAULT NOW(),
    is_cancelled        BOOLEAN DEFAULT false,
    CONSTRAINT valid_time CHECK (end_time > start_time)
);
-- 5. Final Bookings (only after admin approval)
CREATE TABLE bookings (
    booking_id     SERIAL PRIMARY KEY,
    request_id     INTEGER NOT NULL UNIQUE REFERENCES requests(request_id) ON DELETE RESTRICT,
    hall_id        INTEGER NOT NULL REFERENCES halls(hall_id),
    requester_id   INTEGER NOT NULL REFERENCES requesters(requester_id),
    event_title    VARCHAR(200) NOT NULL,
    event_date     DATE NOT NULL,
    start_time     TIME NOT NULL,
    end_time       TIME NOT NULL,
    status         BOOKING_STATUS DEFAULT 'confirmed',
    created_at     TIMESTAMPTZ DEFAULT NOW(),
    completed_at   TIMESTAMPTZ,
    CONSTRAINT valid_booking_time CHECK (end_time > start_time)
);
-- Checks: No other active booking in same hall/date where times overlap
CREATE OR REPLACE FUNCTION check_booking_overlap()
RETURNS TRIGGER AS $$
DECLARE
    overlap_count INTEGER;
BEGIN
    -- Query for overlaps (exclude self for updates, ignore cancelled)
    SELECT COUNT(*)
    INTO overlap_count
    FROM bookings b
    WHERE b.hall_id = NEW.hall_id
      AND b.event_date = NEW.event_date
      AND b.status != 'cancelled'
      AND (
          (NEW.start_time < b.end_time AND NEW.end_time > b.start_time)
      )
      AND b.booking_id != COALESCE(NEW.booking_id, 0);  -- Exclude current row

    IF overlap_count > 0 THEN
        RAISE EXCEPTION 'Booking overlaps with an existing booking on hall % for date %. Time conflict detected.',
NEW.hall_id, NEW.event_date;
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
-- Attach trigger to bookings table
CREATE TRIGGER trig_check_booking_overlap
    BEFORE INSERT OR UPDATE OF start_time, end_time, event_date, hall_id, status
    ON bookings
    FOR EACH ROW
    EXECUTE FUNCTION check_booking_overlap();
-- 6. Audit & Session Management
CREATE TABLE booking_audit_log (
    log_id      SERIAL PRIMARY KEY,
    booking_id  INTEGER REFERENCES bookings(booking_id) ON DELETE SET NULL,
    request_id  INTEGER REFERENCES requests(request_id),
    action      VARCHAR(50) NOT NULL,
    acted_by    INTEGER NOT NULL REFERENCES users(user_id),
    acted_at    TIMESTAMPTZ DEFAULT NOW(),
    remarks     TEXT,
    ip_address  INET
);

CREATE TABLE login_sessions (
    session_id     VARCHAR(255) PRIMARY KEY,
    user_id        INTEGER NOT NULL REFERENCES users(user_id) ON DELETE CASCADE,
    ip_address     INET,
    user_agent     TEXT,
    login_at       TIMESTAMPTZ DEFAULT NOW(),
    last_activity  TIMESTAMPTZ DEFAULT NOW(),
    logout_at      TIMESTAMPTZ,
    is_active      BOOLEAN GENERATED ALWAYS AS (logout_at IS NULL) STORED
);

CREATE TABLE notifications (
    notif_id    SERIAL PRIMARY KEY,
    user_id     INTEGER NOT NULL REFERENCES users(user_id) ON DELETE CASCADE,
    title       VARCHAR(150),
    message     TEXT NOT NULL,
    notif_type  VARCHAR(30) DEFAULT 'info',
```

```
    is_read     BOOLEAN DEFAULT false,
    created_at  TIMESTAMPTZ DEFAULT NOW()
);
-- 7. Indexes for Performance (helps with overlap queries too)
CREATE INDEX idx_bookings_overlap_check ON bookings (hall_id, event_date, start_time, end_time) WHERE status !=
'cancelled';
CREATE INDEX idx_requests_date_hall ON requests(event_date, hall_id);
CREATE INDEX idx_requests_requester ON requests(requester_id);
CREATE INDEX idx_requests_status ON requests(dept_status, admin_status);
CREATE INDEX idx_login_sessions_user ON login_sessions(user_id) WHERE is_active = true;

-- 8. Trigger: Auto-update last_activity on session updates
CREATE OR REPLACE FUNCTION update_session_activity()
RETURNS TRIGGER AS $$
BEGIN
    NEW.last_activity = NOW();
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trig_update_session_activity
    BEFORE UPDATE ON login_sessions
    FOR EACH ROW
    EXECUTE FUNCTION update_session_activity();
```

Backend source (Go)

Seminar Hall Booking System — key Go segments (auth, requests, approval; full repo on GitHub)

backend/main.go

```
package main

import (
	"log"
	"os"

	"seminar-hall-booking-system/internal/config"
	"seminar-hall-booking-system/internal/database"
	"seminar-hall-booking-system/internal/router"

	"github.com/gin-gonic/gin"
	"github.com/joho/godotenv"
)

func main() {
	// Load environment variables (.env only — not .env.production; that file is for deploy docs / hosts)
	_ = godotenv.Load(".env")
	_ = godotenv.Overload(".env.local")

	// Load configuration
	cfg := config.Load()

	// Initialize database connection
	db, err := database.NewConnection(cfg.DatabaseURL)
	if err != nil {
		log.Fatalf("Failed to connect to database: %v", err)
	}
	defer db.Close()

	// Set Gin mode
	if cfg.Environment == "production" {
		gin.SetMode(gin.ReleaseMode)
	}

	// Initialize router
	r := router.SetupRouter(db, cfg)

	// Start server
	port := os.Getenv("PORT")
	if port == "" {
		port = "8080"
	}

	log.Printf("Server starting on port %s", port)
	if err := r.Run(":" + port); err != nil {
		log.Fatalf("Failed to start server: %v", err)
	}
}
```

backend/internal/config/config.go

```
package config

import (
	"os"
)

type Config struct {
	DatabaseURL  string
	JWTSecret    string
	Environment  string
	FrontendURL  string
	Port         string
}

func Load() *Config {
	return &Config{
		DatabaseURL: getEnv("DATABASE_URL", "postgresql://localhost:5432/seminar_hall_db?sslmode=disable"),
		JWTSecret:    getEnv("JWT_SECRET", "your-secret-key-change-in-production"),
		Environment:  getEnv("ENVIRONMENT", "development"),
		FrontendURL:  getEnv("FRONTEND_URL", "http://localhost:3000"),
		Port:         getEnv("PORT", "8080"),
	}
}

func getEnv(key, defaultValue string) string {
	if value := os.Getenv(key); value != "" {
		return value
	}
	return defaultValue
}
```

backend/internal/database/database.go

```
package database

import (
```

```
        "context"
        "fmt"

        "github.com/jackc/pgx/v5"
        "github.com/jackc/pgx/v5/pgxpool"
)

// NewConnection creates a new PostgreSQL connection pool
func NewConnection(databaseURL string) (*pgxpool.Pool, error) {
        config, err := pgxpool.ParseConfig(databaseURL)
        if err != nil {
                return nil, fmt.Errorf("failed to parse database URL: %w", err)
        }

        // Explicitly set search_path to shbs on every new connection.
        // pgx does NOT honour the search_path URL parameter, so we must do it here.
        config.AfterConnect = func(ctx context.Context, conn *pgx.Conn) error {
                _, err := conn.Exec(ctx, "SET search_path TO shbs")
                return err
        }

        pool, err := pgxpool.NewWithConfig(context.Background(), config)
        if err != nil {
                return nil, fmt.Errorf("failed to create connection pool: %w", err)
        }

        // Test connection
        if err := pool.Ping(context.Background()); err != nil {
                return nil, fmt.Errorf("failed to ping database: %w", err)
        }

        return pool, nil
}
```

backend/internal/router/router.go

```
package router

import (
        "seminar-hall-booking-system/internal/config"
        "seminar-hall-booking-system/internal/handlers"
        "seminar-hall-booking-system/internal/middleware"
        "seminar-hall-booking-system/internal/repositories"
        "seminar-hall-booking-system/internal/services"

        "github.com/gin-gonic/gin"
        "github.com/jackc/pgx/v5/pgxpool"
)

func SetupRouter(db *pgxpool.Pool, cfg *config.Config) *gin.Engine {
        r := gin.Default()

        // CORS middleware
        r.Use(middleware.CORS(cfg.FrontendURL))

        // Health check
        r.GET("/health", func(c *gin.Context) {
                c.JSON(200, gin.H{"status": "ok"})
        })

        // Initialize Repositories
        userRepo := repositories.NewUserRepository(db)
        hallRepo := repositories.NewHallRepository(db)
        requestRepo := repositories.NewRequestRepository(db)
        bookingRepo := repositories.NewBookingRepository(db)
        mgmtRepo := repositories.NewManagementRepository(db)

        // Initialize Services
        authService := services.NewAuthService(userRepo, cfg.JWTSecret)
        hallService := services.NewHallService(hallRepo)
        requestService := services.NewRequestService(requestRepo, hallRepo, userRepo, bookingRepo)
        coordinatorService := services.NewCoordinatorService(requestRepo, userRepo, bookingRepo)
        adminService := services.NewAdminService(requestRepo, bookingRepo)

        // Initialize Handlers
        authHandler := handlers.NewAuthHandler(authService, userRepo)
        hallHandler := handlers.NewHallHandler(hallService)
        requestHandler := handlers.NewRequestHandler(requestService)
        coordinatorHandler := handlers.NewCoordinatorHandler(coordinatorService)
        adminHandler := handlers.NewAdminHandler(adminService)
        mgmtHandler := handlers.NewManagementHandler(mgmtRepo)

        // API routes
        api := r.Group("/api")
        {
                // Public routes
                public := api.Group("")
                {
                        public.GET("/halls", hallHandler.GetAllHalls)
                        public.GET("/halls/:id", hallHandler.GetHallByID)
                        public.GET("/availability", hallHandler.GetAvailability)
```

```
                        public.GET("/departments", mgmtHandler.GetDepartments)
                        public.GET("/clubs", mgmtHandler.GetClubs)
                }

                // Auth routes
                auth := api.Group("/auth")
                {
                        auth.POST("/login", authHandler.Login)
                        auth.POST("/logout", authHandler.Logout)

                        protectedAuth := auth.Group("")
                        protectedAuth.Use(middleware.AuthMiddleware(cfg.JWTSecret))
                        protectedAuth.GET("/me", authHandler.GetMe)
                        protectedAuth.PATCH("/password", authHandler.ChangePassword)
                }

                // Protected routes
                protected := api.Group("")
                protected.Use(middleware.AuthMiddleware(cfg.JWTSecret))
                {
                        // Requester routes
                        requester := protected.Group("/requests")
                        requester.Use(middleware.RequireRole("requester"))
                        {
                                requester.POST("", requestHandler.CreateRequest)
                                requester.GET("/my", requestHandler.GetMyRequests)
                        }

                        // Dept Coordinator routes
                        coordinator := protected.Group("")
                        coordinator.Use(middleware.RequireRole("dept_coordinator"))
                        {
                                coordinator.GET("/requests/dept-pending", coordinatorHandler.GetPendingRequests)
                                coordinator.PATCH("/requests/:id/dept-review", coordinatorHandler.ReviewRequest)
                                coordinator.GET("/users/faculties", mgmtHandler.GetFaculties)
                        }

                        // Class management — accessible by both admin and dept_coordinator
                        classGroup := protected.Group("")
                        classGroup.Use(middleware.RequireRole("admin", "dept_coordinator"))
                        {
                                classGroup.GET("/classes", mgmtHandler.GetClasses)
                                classGroup.POST("/classes", mgmtHandler.CreateClass)
                                classGroup.DELETE("/classes/:id", mgmtHandler.DeleteClass)
                        }

                        // Admin routes
                        admin := protected.Group("")
                        admin.Use(middleware.RequireRole("admin"))
                        {
                                // Request & Booking management
                                admin.GET("/requests/admin-pending", adminHandler.GetPendingRequests)
                                admin.PATCH("/requests/:id/admin-review", adminHandler.ReviewRequest)
                                admin.GET("/bookings", adminHandler.GetAllBookings)
                                admin.PATCH("/bookings/:id/cancel", mgmtHandler.CancelBooking)
                                // Department management
                                admin.POST("/departments", mgmtHandler.CreateDepartment)
                                admin.PUT("/departments/:id", mgmtHandler.UpdateDepartment)
                                admin.DELETE("/departments/:id", mgmtHandler.DeleteDepartment)
                                // Hall management
                                admin.POST("/halls", mgmtHandler.CreateHall)
                                admin.PUT("/halls/:id", mgmtHandler.UpdateHall)
                                admin.DELETE("/halls/:id", mgmtHandler.DeleteHall)
                                // User management
                                admin.GET("/users", mgmtHandler.GetAllUsers)
                                admin.POST("/users", mgmtHandler.CreateUser)
                                admin.DELETE("/users/:id", mgmtHandler.DeactivateUser)
                                admin.PATCH("/users/:id/reactivate", mgmtHandler.ReactivateUser)
                                admin.PATCH("/users/:id/password", mgmtHandler.UpdateUserPassword)
                                // Club management (write-only; GET /clubs is public)
                                admin.POST("/clubs", mgmtHandler.CreateClub)
                                admin.DELETE("/clubs/:id", mgmtHandler.DeleteClub)
                                // Reports
                                admin.GET("/reports/hall-usage", mgmtHandler.GetHallUsageReport)
                                admin.GET("/reports/department-usage", mgmtHandler.GetDeptUsageReport)
                        }
                }
        }

        return r
}
```

backend/internal/middleware/auth.go

```
package middleware

import (
        "net/http"
        "strings"

        "github.com/gin-gonic/gin"
        "github.com/golang-jwt/jwt/v5"
```

```
)

// AuthMiddleware validates JWT token from cookie or Authorization header
func AuthMiddleware(jwtSecret string) gin.HandlerFunc {
	return func(c *gin.Context) {
		// Try to get token from cookie first
		tokenString, err := c.Cookie("auth_token")
		if err != nil {
			// Try Authorization header
			authHeader := c.GetHeader("Authorization")
			if authHeader == "" {
				c.JSON(http.StatusUnauthorized, gin.H{"error": "Unauthorized"})
				c.Abort()
				return
			}

			// Extract token from "Bearer <token>"
			parts := strings.Split(authHeader, " ")
			if len(parts) != 2 || parts[0] != "Bearer" {
				c.JSON(http.StatusUnauthorized, gin.H{"error": "Invalid authorization header"})
				c.Abort()
				return
			}
			tokenString = parts[1]
		}

		// Parse and validate token
		token, err := jwt.Parse(tokenString, func(token *jwt.Token) (interface{}, error) {
			if _, ok := token.Method.(*jwt.SigningMethodHMAC); !ok {
				return nil, jwt.ErrSignatureInvalid
			}
			return []byte(jwtSecret), nil
		})

		if err != nil || !token.Valid {
			c.JSON(http.StatusUnauthorized, gin.H{"error": "Invalid token"})
			c.Abort()
			return
		}

		// Extract claims
		if claims, ok := token.Claims.(jwt.MapClaims); ok {
			c.Set("user_id", int(claims["user_id"].(float64)))
			c.Set("username", claims["username"].(string))
			c.Set("role", claims["role"].(string))
		}

		c.Next()
	}
}

// RequireRole checks if user has one of the required roles
func RequireRole(allowedRoles ...string) gin.HandlerFunc {
	return func(c *gin.Context) {
		userRole, exists := c.Get("role")
		if !exists {
			c.JSON(http.StatusForbidden, gin.H{"error": "Insufficient permissions"})
			c.Abort()
			return
		}
		for _, r := range allowedRoles {
			if userRole.(string) == r {
				c.Next()
				return
			}
		}
		c.JSON(http.StatusForbidden, gin.H{"error": "Insufficient permissions"})
		c.Abort()
	}
}
```

backend/internal/models/user.go

```
package models

import (
	"time"
)

type UserRole string

const (
	RoleAdmin           UserRole = "admin"
	RoleDeptCoordinator UserRole = "dept_coordinator"
	RoleRequester       UserRole = "requester"
	RoleFaculty         UserRole = "faculty"
)

type User struct {
	UserID      int       `json:"user_id"`
	Username    string    `json:"username"`
```

```
        PasswordHash string   `json:"-"` // Never return in JSON
        FullName    string    `json:"full_name"`
        Email       *string   `json:"email"`
        Phone       *string   `json:"phone"`
        Role        UserRole  `json:"role"`
        DeptID      *int      `json:"dept_id"`
        IsActive    bool      `json:"is_active"`
        CreatedAt   time.Time `json:"created_at"`
        UpdatedAt   time.Time `json:"updated_at"`
}

type RequesterType string

const (
        RequesterTypeClass RequesterType = "class"
        RequesterTypeClub  RequesterType = "club"
)

type Requester struct {
        RequesterID   int           `json:"requester_id"`
        UserID        int           `json:"user_id"`
        RequesterType RequesterType `json:"requester_type"`
        ClassID       *int          `json:"class_id"`
        ClubID        *int          `json:"club_id"`
        CreatedAt     time.Time     `json:"created_at"`
}
```

backend/internal/models/booking.go (excerpt: L1–73)

```
package models

import (
        "time"
)

type ApprovalStatus string

const (
        StatusPending   ApprovalStatus = "pending"
        StatusForwarded ApprovalStatus = "forwarded"
        StatusApproved  ApprovalStatus = "approved"
        StatusRejected  ApprovalStatus = "rejected"
        StatusAmended   ApprovalStatus = "amended"
)

type BookingStatus string

const (
        BookingStatusConfirmed BookingStatus = "confirmed"
        BookingStatusCancelled BookingStatus = "cancelled"
        BookingStatusCompleted BookingStatus = "completed"
        BookingStatusNoShow    BookingStatus = "no_show"
)

type Hall struct {
        HallID     int                    `json:"hall_id"`
        HallName   string                 `json:"hall_name"`
        Capacity   int                    `json:"capacity"`
        Location   *string                `json:"location"`
        Facilities map[string]interface{} `json:"facilities"`
        IsActive   bool                   `json:"is_active"`
}

type Request struct {
        RequestID         int            `json:"request_id"`
        RequesterID       int            `json:"requester_id"`
        HallID            int            `json:"hall_id"`
        HallName          *string        `json:"hall_name,omitempty"`
        HallLocation      *string        `json:"hall_location,omitempty"`
        HallCapacity      *int           `json:"hall_capacity,omitempty"`
        EventTitle        string         `json:"event_title"`
        EventDescription  *string        `json:"event_description"`
        EventDate         time.Time      `json:"event_date"`
        StartTime         string         `json:"start_time"` // TIME format
        EndTime           string         `json:"end_time"`   // TIME format
        ExpectedAttendees *int           `json:"expected_attendees"`
        Purpose           *string        `json:"purpose"`
        DeptStatus        ApprovalStatus `json:"dept_status"`
        DeptApprovedBy    *int           `json:"dept_approved_by"`
        DeptApprovedAt    *time.Time     `json:"dept_approved_at"`
        DeptRemarks       *string        `json:"dept_remarks"`
        AdminStatus       ApprovalStatus `json:"admin_status"`
        AdminApprovedBy   *int           `json:"admin_approved_by"`
        AdminApprovedAt   *time.Time     `json:"admin_approved_at"`
        AdminRemarks      *string        `json:"admin_remarks"`
        RequestedAt       time.Time      `json:"requested_at"`
        IsCancelled       bool           `json:"is_cancelled"`
}

type Booking struct {
        BookingID   int           `json:"booking_id"`
```

```
        RequestID   int           `json:"request_id"`
        HallID      int           `json:"hall_id"`
        RequesterID int           `json:"requester_id"`
        EventTitle  string        `json:"event_title"`
        EventDate   time.Time     `json:"event_date"`
        StartTime   string        `json:"start_time"`
        EndTime     string        `json:"end_time"`
        Status      BookingStatus `json:"status"`
        CreatedAt   time.Time     `json:"created_at"`
        CompletedAt *time.Time    `json:"completed_at"`
}
// ...
```

backend/internal/handlers/auth_handler.go (excerpt: L1–87)

```
package handlers

import (
        "errors"
        "net/http"
        "seminar-hall-booking-system/internal/repositories"
        "seminar-hall-booking-system/internal/services"
        "strings"

        "github.com/gin-gonic/gin"
        "github.com/jackc/pgx/v5"
)

type AuthHandler struct {
        authService *services.AuthService
        userRepo    *repositories.UserRepository
}

func NewAuthHandler(authService *services.AuthService, userRepo *repositories.UserRepository) *AuthHandler {
        return &AuthHandler{
                authService: authService,
                userRepo:    userRepo,
        }
}

type LoginRequest struct {
        Username string `json:"username" binding:"required"`
        Password string `json:"password" binding:"required"`
}

// Login handles user authentication
func (h *AuthHandler) Login(c *gin.Context) {
        var req LoginRequest
        if err := c.ShouldBindJSON(&req); err != nil {
                c.JSON(http.StatusBadRequest, gin.H{"error": err.Error()})
                return
        }

        token, user, err := h.authService.Login(c.Request.Context(), req.Username, req.Password)
        if err != nil {
                c.JSON(http.StatusUnauthorized, gin.H{"error": "invalid credentials"})
                return
        }

        // Set HTTP-only cookie for the JWT
        c.SetCookie("auth_token", token, 3600*24*7, "/", "", false, true)

        c.JSON(http.StatusOK, gin.H{
                "message": "success",
                "token":   token,
                "user":    user,
        })
}

// Logout clears the auth cookie
func (h *AuthHandler) Logout(c *gin.Context) {
        c.SetCookie("auth_token", "", -1, "/", "", false, true)
        c.JSON(http.StatusOK, gin.H{"message": "logged out"})
}

// GetMe returns the authenticated user's details
func (h *AuthHandler) GetMe(c *gin.Context) {
        userID, exists := c.Get("user_id")
        if !exists {
                c.JSON(http.StatusUnauthorized, gin.H{"error": "unauthorized"})
                return
        }

        // Fetch fresh user details from DB
        user, err := h.userRepo.GetByID(c.Request.Context(), userID.(int))
        if err != nil || user == nil {
                c.JSON(http.StatusUnauthorized, gin.H{"error": "user not found"})
                return
        }

        // Check if user is a requester and fetch requester info
        // if user.Role == models.RoleRequester
```

```
        if user.Role == "requester" {
                reqInfo, err := h.userRepo.GetRequesterByUserID(c.Request.Context(), user.UserID)
                if err == nil && reqInfo != nil {
                        c.JSON(http.StatusOK, gin.H{"user": user, "requester": reqInfo})
                        return
                }
        }

        c.JSON(http.StatusOK, gin.H{"user": user})
}
// ...
```

backend/internal/services/auth_service.go (excerpt: L1–63)

```
package services

import (
        "context"
        "errors"
        "fmt"
        "seminar-hall-booking-system/internal/models"
        "seminar-hall-booking-system/internal/repositories"
        "time"

        "github.com/golang-jwt/jwt/v5"
        "golang.org/x/crypto/bcrypt"
)

// ErrInvalidCurrentPassword is returned when ChangeOwnPassword verification fails.
var ErrInvalidCurrentPassword = errors.New("invalid current password")

type AuthService struct {
        userRepo  *repositories.UserRepository
        jwtSecret string
}

func NewAuthService(userRepo *repositories.UserRepository, jwtSecret string) *AuthService {
        return &AuthService{
                userRepo:  userRepo,
                jwtSecret: jwtSecret,
        }
}

func (s *AuthService) Login(ctx context.Context, username, password string) (string, *models.User, error) {
        user, err := s.userRepo.GetByUsername(ctx, username)
        if err != nil {
                return "", nil, err
        }
        if user == nil {
                return "", nil, errors.New("invalid credentials")
        }

        // Attempt bcrypt comparison
        err = bcrypt.CompareHashAndPassword([]byte(user.PasswordHash), []byte(password))
        if err != nil {
                // Fallback for sample DB data which might be plain text or pseudo-hashed strings like
"hashedpass_admin"
                if user.PasswordHash != password {
                        return "", nil, errors.New("invalid credentials")
                }
        }

        // Generate JWT claims
        token := jwt.NewWithClaims(jwt.SigningMethodHS256, jwt.MapClaims{
                "user_id":  user.UserID,
                "username": user.Username,
                "role":     user.Role,
                "exp":      time.Now().Add(time.Hour * 24 * 7).Unix(), // 7 days expiration
        })

        // Sign token
        tokenString, err := token.SignedString([]byte(s.jwtSecret))
        if err != nil {
                return "", nil, err
        }

        return tokenString, user, nil
}
// ...
```

backend/internal/handlers/request_handler.go (excerpt: L1–85)

```
package handlers

import (
        "fmt"
        "net/http"
        "seminar-hall-booking-system/internal/models"
        "seminar-hall-booking-system/internal/services"
        "time"

        "github.com/gin-gonic/gin"
)
```

```
type RequestHandler struct {
	requestService *services.RequestService
}

func NewRequestHandler(requestService *services.RequestService) *RequestHandler {
	return &RequestHandler{requestService: requestService}
}

type CreateRequestPayload struct {
	HallID            int     `json:"hall_id" binding:"required"`
	EventTitle        string  `json:"event_title" binding:"required"`
	EventDescription  *string `json:"event_description"`
	EventDate         string  `json:"event_date" binding:"required"` // YYYY-MM-DD
	StartTime         string  `json:"start_time" binding:"required"` // HH:MM
	EndTime           string  `json:"end_time" binding:"required"`   // HH:MM
	ExpectedAttendees *int    `json:"expected_attendees"`
	Purpose           *string `json:"purpose"`
}

// CreateRequest creates a new booking request for a student/club
func (h *RequestHandler) CreateRequest(c *gin.Context) {
	userIDStr, exists := c.Get("user_id")
	if !exists {
		c.JSON(http.StatusUnauthorized, gin.H{"error": "unauthorized"})
		return
	}
	userID := userIDStr.(int)

	var payload CreateRequestPayload
	if err := c.ShouldBindJSON(&payload); err != nil {
		c.JSON(http.StatusBadRequest, gin.H{"error": err.Error()})
		return
	}

	// Parse date
	eventDate, err := time.Parse("2006-01-02", payload.EventDate)
	if err != nil {
		c.JSON(http.StatusBadRequest, gin.H{"error": "invalid event_date format (use YYYY-MM-DD)"})
		return
	}

	// Format times string slightly (adding seconds if omitted by client)
	startTime := payload.StartTime
	if len(startTime) == 5 {
		startTime += ":00"
	}
	endTime := payload.EndTime
	if len(endTime) == 5 {
		endTime += ":00"
	}

	req := models.Request{
		HallID:            payload.HallID,
		EventTitle:        payload.EventTitle,
		EventDescription:  payload.EventDescription,
		EventDate:         eventDate,
		StartTime:         startTime,
		EndTime:           endTime,
		ExpectedAttendees: payload.ExpectedAttendees,
		Purpose:           payload.Purpose,
	}

	err = h.requestService.CreateRequest(c.Request.Context(), userID, &req)
	if err != nil {
		c.JSON(http.StatusBadRequest, gin.H{"error": err.Error()})
		return
	}

	c.JSON(http.StatusCreated, gin.H{
		"message": "request submitted successfully",
		"data":    req,
	})
}
// ...
```

backend/internal/services/request_service.go (excerpt: L1–86)

```
package services

import (
	"context"
	"errors"
	"seminar-hall-booking-system/internal/models"
	"seminar-hall-booking-system/internal/repositories"
	"time"
)

type RequestService struct {
	requestRepo *repositories.RequestRepository
	hallRepo    *repositories.HallRepository
	userRepo    *repositories.UserRepository
	bookingRepo *repositories.BookingRepository
```

```
}

func NewRequestService(
        reqRepo *repositories.RequestRepository,
        hallRepo *repositories.HallRepository,
        userRepo *repositories.UserRepository,
        bookingRepo *repositories.BookingRepository,
) *RequestService {
        return &RequestService{
                requestRepo: reqRepo,
                hallRepo:    hallRepo,
                userRepo:    userRepo,
                bookingRepo: bookingRepo,
        }
}

// CreateRequest handles the business logic for submitting a new seminar hall request
func (s *RequestService) CreateRequest(ctx context.Context, userID int, req *models.Request) error {
        // 1. Verify user is a requester
        requester, err := s.userRepo.GetRequesterByUserID(ctx, userID)
        if err != nil {
                return err
        }
        if requester == nil {
                return errors.New("user is not registered as a requester (class or club)")
        }

        // 2. Validate basic rules
        if req.ExpectedAttendees != nil && *req.ExpectedAttendees <= 0 {
                return errors.New("expected attendees must be greater than zero")
        }

        // 3. Verify Hall exists and capacity is sufficient
        hall, err := s.hallRepo.GetByID(ctx, req.HallID)
        if err != nil {
                return err
        }
        if hall == nil {
                return errors.New("hall not found")
        }
        if req.ExpectedAttendees != nil && *req.ExpectedAttendees > hall.Capacity {
                return errors.New("expected attendees exceeds hall capacity")
        }

        // 4. Validate time (end_time > start_time)
        startTime, err := time.Parse("15:04:05", req.StartTime)
        if err != nil {
                return errors.New("invalid start_time format, expected HH:MM:SS")
        }
        endTime, err := time.Parse("15:04:05", req.EndTime)
        if err != nil {
                return errors.New("invalid end_time format, expected HH:MM:SS")
        }
        if !endTime.After(startTime) {
                return errors.New("end time must be strictly after start time")
        }

        // 5. Check for overlapping bookings before placing the request
        overlap, err := s.bookingRepo.CheckOverlap(ctx, req.HallID, req.EventDate, req.StartTime, req.EndTime)
        if err != nil {
                return err
        }
        if overlap {
                return errors.New("cannot place request: this time slot overlaps with an existing confirmed
booking")
        }

        // 6. Build and save request
        req.RequesterID = requester.RequesterID

        return s.requestRepo.Create(ctx, req)
}
// ...
```

backend/internal/repositories/request_repository.go (excerpt: L1–40, L145–153)

```
package repositories

import (
        "context"
        "seminar-hall-booking-system/internal/models"

        "github.com/jackc/pgx/v5"
        "github.com/jackc/pgx/v5/pgxpool"
)

type RequestRepository struct {
        db *pgxpool.Pool
}

func NewRequestRepository(db *pgxpool.Pool) *RequestRepository {
        return &RequestRepository{db: db}
```

```
}

// Create new request
func (r *RequestRepository) Create(ctx context.Context, req *models.Request) error {
        query := `
                INSERT INTO requests (requester_id, hall_id, event_title, event_description, event_date,
start_time, end_time, expected_attendees, purpose)
                VALUES ($1, $2, $3, $4, $5, $6, $7, $8, $9)
                RETURNING request_id, dept_status, admin_status, requested_at
        `

        err := r.db.QueryRow(ctx, query,
                req.RequesterID,
                req.HallID,
                req.EventTitle,
                req.EventDescription,
                req.EventDate,
                req.StartTime,
                req.EndTime,
                req.ExpectedAttendees,
                req.Purpose,
        ).Scan(&req.RequestID, &req.DeptStatus, &req.AdminStatus, &req.RequestedAt)

        return err
}
// ...
func (r *RequestRepository) UpdateDeptStatus(ctx context.Context, reqID int, status models.ApprovalStatus, byUserID
int, remarks *string) error {
        query := `
                UPDATE requests
                SET dept_status = $1, dept_approved_by = $2, dept_approved_at = NOW(), dept_remarks = $3
                WHERE request_id = $4
        `
        _, err := r.db.Exec(ctx, query, status, byUserID, remarks, reqID)
        return err
}
// ...
```

backend/internal/handlers/coordinator_handler.go

```
package handlers

import (
        "net/http"
        "seminar-hall-booking-system/internal/models"
        "seminar-hall-booking-system/internal/services"
        "strconv"

        "github.com/gin-gonic/gin"
)

type CoordinatorHandler struct {
        coordinatorService *services.CoordinatorService
}

func NewCoordinatorHandler(coordinatorService *services.CoordinatorService) *CoordinatorHandler {
        return &CoordinatorHandler{coordinatorService: coordinatorService}
}

// GetPendingRequests gets requests waiting for department approval
func (h *CoordinatorHandler) GetPendingRequests(c *gin.Context) {
        userIDStr, exists := c.Get("user_id")
        if !exists {
                c.JSON(http.StatusUnauthorized, gin.H{"error": "unauthorized"})
                return
        }
        userID := userIDStr.(int)

        requests, err := h.coordinatorService.GetDepartmentPendingRequests(c.Request.Context(), userID)
        if err != nil {
                c.JSON(http.StatusInternalServerError, gin.H{"error": err.Error()})
                return
        }

        if requests == nil {
                requests = []models.Request{}
        }

        c.JSON(http.StatusOK, requests)
}

type ReviewRequestPayload struct {
        Action  string  `json:"action" binding:"required"` // "approve" or "reject"
        Remarks *string `json:"remarks"`
}

// ReviewRequest approves or rejects a department request
func (h *CoordinatorHandler) ReviewRequest(c *gin.Context) {
        userIDStr, exists := c.Get("user_id")
        if !exists {
                c.JSON(http.StatusUnauthorized, gin.H{"error": "unauthorized"})
                return
```

```
        }
        userID := userIDStr.(int)

        requestIDStr := c.Param("id")
        requestID, err := strconv.Atoi(requestIDStr)
        if err != nil {
                c.JSON(http.StatusBadRequest, gin.H{"error": "invalid request id"})
                return
        }

        var payload ReviewRequestPayload
        if err := c.ShouldBindJSON(&payload); err != nil {
                c.JSON(http.StatusBadRequest, gin.H{"error": err.Error()})
                return
        }

        err = h.coordinatorService.ReviewRequest(c.Request.Context(), userID, requestID, payload.Action,
payload.Remarks)
        if err != nil {
                c.JSON(http.StatusBadRequest, gin.H{"error": err.Error()})
                return
        }

        c.JSON(http.StatusOK, gin.H{"message": "request " + payload.Action + " successfully"})
}
```

backend/internal/services/coordinator_service.go (excerpt: L1–22, L40–79)

```
package services

import (
        "context"
        "errors"
        "seminar-hall-booking-system/internal/models"
        "seminar-hall-booking-system/internal/repositories"
)

type CoordinatorService struct {
        requestRepo *repositories.RequestRepository
        userRepo    *repositories.UserRepository
        bookingRepo *repositories.BookingRepository
}

func NewCoordinatorService(reqRepo *repositories.RequestRepository, userRepo *repositories.UserRepository,
bookingRepo *repositories.BookingRepository) *CoordinatorService {
        return &CoordinatorService{
                requestRepo: reqRepo,
                userRepo:    userRepo,
                bookingRepo: bookingRepo,
        }
}
// ...
// ReviewRequest allows the coordinator to approve (forward) or reject a request
func (s *CoordinatorService) ReviewRequest(ctx context.Context, coordinatorID int, requestID int, action string,
remarks *string) error {
        // 1. Verify coordinator
        coordinator, err := s.userRepo.GetByID(ctx, coordinatorID)
        if err != nil || coordinator == nil || coordinator.DeptID == nil {
                return errors.New("invalid coordinator")
        }

        // 2. Verify request exists and is pending
        req, err := s.requestRepo.GetByID(ctx, requestID)
        if err != nil || req == nil {
                return errors.New("request not found")
        }

        if req.DeptStatus != models.StatusPending {
                return errors.New("request is no longer pending department approval")
        }

        // In a full implementation, verify the request belongs to the coordinator's department here by looking up
the requester's department.

        // 3. Update status
        var newStatus models.ApprovalStatus
        if action == "approve" {
                // Before forwarding, check if there's an existing overlapping booking
                overlap, err := s.bookingRepo.CheckOverlap(ctx, req.HallID, req.EventDate, req.StartTime,
req.EndTime)
                if err != nil {
                        return err
                }
                if overlap {
                        return errors.New("cannot forward: this request overlaps with an existing confirmed
booking")
                }
                newStatus = models.StatusForwarded
        } else if action == "reject" {
                newStatus = models.StatusRejected
        } else {
                return errors.New("invalid action. Must be 'approve' or 'reject'")
```

```
        }

        return s.requestRepo.UpdateDeptStatus(ctx, requestID, newStatus, coordinatorID, remarks)
}
```

backend/internal/repositories/booking_repository.go (excerpt: L1–18, L126–138, L140–145)

```
package repositories

import (
        "context"
        "seminar-hall-booking-system/internal/models"
        "time"

        "github.com/jackc/pgx/v5/pgxpool"
)

type BookingRepository struct {
        db *pgxpool.Pool
}

func NewBookingRepository(db *pgxpool.Pool) *BookingRepository {
        return &BookingRepository{db: db}
}

// ...
// CheckOverlap returns true if there's an existing active booking for the same hall and time
func (r *BookingRepository) CheckOverlap(ctx context.Context, hallID int, date time.Time, startTime string, endTime
string) (bool, error) {
        query := `
                SELECT COUNT(*)
                FROM bookings
                WHERE hall_id = $1
                  AND event_date = $2
                  AND status != 'cancelled'
                  AND (
                          (start_time < $4 AND end_time > $3)
                  )
        `
        var count int
// ...
        err := r.db.QueryRow(ctx, query, hallID, date, startTime, endTime).Scan(&count)
        if err != nil {
                return false, err
        }
        return count > 0, nil
}
```

Frontend source (Next.js / TypeScript)

Seminar Hall Booking System — key Next.js/TS segments (full repo on GitHub)

frontend/lib/api.ts

```
import axios from 'axios';

const API_URL = process.env.NEXT_PUBLIC_API_URL || 'http://localhost:8080';

export const api = axios.create({
  baseURL: `${API_URL}/api`,
  headers: {
    'Content-Type': 'application/json',
  },
});

// Attach JWT token from localStorage on every request
api.interceptors.request.use((config) => {
  if (typeof window !== 'undefined') {
    const token = localStorage.getItem('auth_token');
    if (token) {
      config.headers.Authorization = `Bearer ${token}`;
    }
  }
  return config;
});

// Response interceptor for error handling
api.interceptors.response.use(
  (response) => response,
  (error) => {
    if (error.response?.status === 401) {
      // Only redirect if we're on a protected page (not on login or public pages)
      if (typeof window !== 'undefined') {
        const publicPaths = ['/login', '/', '/halls'];
        const isPublic = publicPaths.some((p) =>
          window.location.pathname === p || window.location.pathname.startsWith(`${p}/`)
        );
        if (!isPublic) {
          localStorage.removeItem('auth_token');
          window.location.href = '/login';
        }
      }
    }
    return Promise.reject(error);
  }
);
```

frontend/lib/auth-context.tsx

```
'use client';

import { createContext, useContext, useEffect, useState, ReactNode } from 'react';
import { authApi } from '@/lib/services';
import { User } from '@/lib/types';

interface AuthContextType {
  user: User | null;
  requesterInfo: { requester_id: number; requester_type: string } | null;
  loading: boolean;
  logout: () => Promise<void>;
  refresh: () => Promise<void>;
}

const AuthContext = createContext<AuthContextType | null>(null);

export function AuthProvider({ children }: { children: ReactNode }) {
  const [user, setUser] = useState<User | null>(null);
  const [requesterInfo, setRequesterInfo] = useState<{ requester_id: number; requester_type: string } | null>
(null);
  const [loading, setLoading] = useState(true);

  const refresh = async () => {
    try {
      const res = await authApi.getMe();
      setUser(res.data.user);
      setRequesterInfo(res.data.requester ?? null);
    } catch {
      setUser(null);
      setRequesterInfo(null);
    } finally {
      setLoading(false);
    }
  };

  const logout = async () => {
    try {
      await authApi.logout();
    } finally {
      localStorage.removeItem('auth_token');
```

```
      setUser(null);
      setRequesterInfo(null);
      window.location.href = '/login';
    }
  };

  useEffect(() => {
    refresh();
  }, []);

  return (
    <AuthContext.Provider value={{ user, requesterInfo, loading, logout, refresh }}>
      {children}
    </AuthContext.Provider>
  );
}

export function useAuth() {
  const ctx = useContext(AuthContext);
  if (!ctx) throw new Error('useAuth must be used inside AuthProvider');
  return ctx;
}
```

frontend/lib/types.ts (excerpt: L1–109)

```
// User and Auth Types
export type UserRole = 'admin' | 'dept_coordinator' | 'requester' | 'faculty';
export type ApprovalStatus = 'pending' | 'forwarded' | 'approved' | 'rejected' | 'amended';
export type BookingStatus = 'confirmed' | 'cancelled' | 'completed' | 'no_show';
export type RequesterType = 'class' | 'club';

export interface User {
  user_id: number;
  username: string;
  full_name: string;
  email?: string;
  phone?: string;
  role: UserRole;
  dept_id?: number;
  is_active: boolean;
  created_at: string;
  updated_at: string;
}

export interface Requester {
  requester_id: number;
  user_id: number;
  requester_type: RequesterType;
  class_id?: number;
  club_id?: number;
  created_at: string;
}

export interface Hall {
  hall_id: number;
  hall_name: string;
  capacity: number;
  location?: string;
  facilities: Record<string, any>;
  is_active: boolean;
}

export interface Request {
  request_id: number;
  requester_id: number;
  hall_id: number;
  hall_name?: string;
  hall_location?: string;
  hall_capacity?: number;
  event_title: string;
  event_description?: string;
  event_date: string;
  start_time: string;
  end_time: string;
  expected_attendees?: number;
  purpose?: string;
  dept_status: ApprovalStatus;
  dept_approved_by?: number;
  dept_approved_at?: string;
  dept_remarks?: string;
  admin_status: ApprovalStatus;
  admin_approved_by?: number;
  admin_approved_at?: string;
  admin_remarks?: string;
  requested_at: string;
  is_cancelled: boolean;
}

export interface Booking {
  booking_id: number;
  request_id: number;
  hall_id: number;
```

```
  requester_id: number;
  event_title: string;
  event_date: string;
  start_time: string;
  end_time: string;
  status: BookingStatus;
  created_at: string;
  completed_at?: string;
}

export interface AvailabilitySlot {
  hall_id: number;
  hall_name: string;
  date: string;
  bookings: Booking[];
}

export interface LoginRequest {
  username: string;
  password: string;
}

export interface LoginResponse {
  user: User;
  token: string;
}

export interface CreateRequestInput {
  hall_id: number;
  event_title: string;
  event_description?: string;
  event_date: string;
  start_time: string;
  end_time: string;
  expected_attendees?: number;
  purpose?: string;
}

export interface ReviewRequestInput {
  action: 'approve' | 'reject';
  remarks?: string;
}
// ...
```

frontend/lib/services.ts (excerpt: L1–30, L70–84)

```
import { api } from './api';
import {
  Hall, Booking, Department, Class, Club,
  HallUsageReport, DepartmentUsageReport,
  Request, User, ReviewRequestInput, CreateRequestInput,
} from './types';

// ── AUTH ──────────────────────────────────────────────────────────────

export const authApi = {
  login: async (username: string, password: string) => {
    const res = await api.post<{ user: User; token: string }>('/auth/login', { username, password });
    // Save token to localStorage so the request interceptor can use it
    if (typeof window !== 'undefined') {
      localStorage.setItem('auth_token', res.data.token);
    }
    return res;
  },

  logout: () => api.post('/auth/logout'),

  getMe: () => api.get<{ user: User; requester?: { requester_id: number; requester_type: string } }>('/auth/me'),

  /** Any logged-in user: verify current password and set a new one */
  changeOwnPassword: (currentPassword: string, newPassword: string) =>
    api.patch('/auth/password', {
      current_password: currentPassword,
      new_password: newPassword,
    }),
};
// ...
// ── REQUESTS ──────────────────────────────────────────────────────────

export const requestsApi = {
  // Requester
  create: (data: CreateRequestInput) => api.post<Request>('/requests', data),
  getMy: () => api.get<Request[]>('/requests/my'),

  // Coordinator
  getDeptPending: () => api.get<Request[]>('/requests/dept-pending'),
  deptReview: (id: number, payload: ReviewRequestInput) => api.patch(`/requests/${id}/dept-review`, payload),

  // Admin
  getAdminPending: () => api.get<Request[]>('/requests/admin-pending'),
```

```
  adminReview: (id: number, payload: ReviewRequestInput) => api.patch(`/requests/${id}/admin-review`, payload),
};
// ...
```

frontend/app/login/page.tsx (excerpt: L1–45)

```
'use client';

import { useState, useEffect } from 'react';
import { useRouter } from 'next/navigation';
import Link from 'next/link';
import { authApi } from '@/lib/services';
import { useAuth } from '@/lib/auth-context';
import { toast } from 'sonner';

export default function LoginPage() {
  const router = useRouter();
  const { user, loading, refresh } = useAuth();
  const [formData, setFormData] = useState({ username: '', password: '' });
  const [submitting, setSubmitting] = useState(false);

  // If already logged in, redirect to dashboard
  useEffect(() => {
    if (!loading && user) {
      router.push('/dashboard');
    }
  }, [user, loading, router]);

  const handleSubmit = async (e: React.FormEvent) => {
    e.preventDefault();
    setSubmitting(true);
    try {
      await authApi.login(formData.username, formData.password);
      // Refresh AuthContext so user state is populated BEFORE navigating
      await refresh();
      toast.success('Login successful!');
      router.push('/dashboard');
    } catch (error: any) {
      toast.error(error.response?.data?.error || 'Login failed');
    } finally {
      setSubmitting(false);
    }
  };

  return (
    <div className="min-h-screen flex items-center justify-center bg-gradient-to-br from-blue-50 to-white">
      <div className="bg-white p-8 rounded-lg shadow-lg w-full max-w-md">
        <h1 className="text-3xl font-bold text-blue-900 mb-6 text-center">
          Login
        </h1>
        <form onSubmit={handleSubmit} className="space-y-4">
// ...
```

frontend/app/requester/request-new/page.tsx (excerpt: L1–55)

```
'use client';

import { useEffect, useState } from 'react';
import { useRouter } from 'next/navigation';
import { useAuth } from '@/lib/auth-context';
import { hallsApi, requestsApi } from '@/lib/services';
import { Hall } from '@/lib/types';
import { toast } from 'sonner';

export default function NewRequestPage() {
  const { user, loading } = useAuth();
  const router = useRouter();
  const [halls, setHalls] = useState<Hall[]>([]);
  const [submitting, setSubmitting] = useState(false);

  const [form, setForm] = useState({
    hall_id: 0,
    event_title: '',
    event_description: '',
    event_date: '',
    start_time: '',
    end_time: '',
    expected_attendees: '',
    purpose: '',
  });

  useEffect(() => {
    if (!loading && !user) router.push('/login');
    if (!loading && user && user.role !== 'requester') router.push('/dashboard');
  }, [user, loading, router]);

  useEffect(() => {
    hallsApi.getAll().then((res) => setHalls(res.data)).catch(() => toast.error('Failed to load halls'));
  }, []);

  const handleSubmit = async (e: React.FormEvent) => {
    e.preventDefault();
    if (!form.hall_id) { toast.error('Please select a hall'); return; }
```

```
    setSubmitting(true);
    try {
      await requestsApi.create({
        hall_id: form.hall_id,
        event_title: form.event_title,
        event_description: form.event_description || undefined,
        event_date: form.event_date,
        start_time: form.start_time,
        end_time: form.end_time,
        expected_attendees: form.expected_attendees ? parseInt(form.expected_attendees) : undefined,
        purpose: form.purpose || undefined,
      });
      toast.success('Request submitted successfully!');
      router.push('/requester/my-requests');
    } catch (err: any) {
      toast.error(err.response?.data?.error || 'Failed to submit request');
// ...
```

frontend/app/coordinator/pending/page.tsx (excerpt: L1–50)

```
'use client';

import { useEffect, useState } from 'react';
import { useRouter } from 'next/navigation';
import { useAuth } from '@/lib/auth-context';
import { requestsApi } from '@/lib/services';
import { Request } from '@/lib/types';
import { formatDate, formatTime } from '@/lib/utils';
import { toast } from 'sonner';

export default function CoordinatorPendingPage() {
  const { user, loading } = useAuth();
  const router = useRouter();
  const [requests, setRequests] = useState<Request[]>([]);
  const [fetching, setFetching] = useState(true);
  const [reviewing, setReviewing] = useState<number | null>(null);

  useEffect(() => {
    if (!loading && !user) router.push('/login');
    if (!loading && user && user.role !== 'dept_coordinator') router.push('/dashboard');
  }, [user, loading, router]);

  const fetchRequests = () => {
    requestsApi.getDeptPending()
      .then((res) => setRequests(res.data))
      .catch(() => toast.error('Failed to load requests'))
      .finally(() => setFetching(false));
  };

  useEffect(() => {
    if (user?.role === 'dept_coordinator') fetchRequests();
  }, [user]);

  const handleReview = async (id: number, action: 'approve' | 'reject') => {
    const remarks = action === 'reject' ? window.prompt('Enter rejection remarks (optional):') ?? undefined :
undefined;
    setReviewing(id);
    try {
      await requestsApi.deptReview(id, { action, remarks });
      toast.success(`Request ${action === 'approve' ? 'forwarded to admin' : 'rejected'}`);
      fetchRequests();
    } catch (err: any) {
      toast.error(err.response?.data?.error || 'Action failed');
    } finally {
      setReviewing(null);
    }
  };

  if (loading || fetching) return <div className="min-h-screen flex items-center justify-center"><div
className="animate-spin rounded-full h-12 w-12 border-b-2 border-blue-600"></div></div>;

  return (
// ...
```

frontend/app/halls/[id]/calendar/page.tsx (excerpt: L1–68)

```
'use client';

import { useState, useEffect } from 'react';
import { useParams, useRouter } from 'next/navigation';
import Link from 'next/link';
import { Calendar, momentLocalizer, View } from 'react-big-calendar';
import moment from 'moment';
import 'react-big-calendar/lib/css/react-big-calendar.css';
import { api } from '@/lib/api';
import { Booking, Hall } from '@/lib/types';
import { formatTime } from '@/lib/utils';
import { toast } from 'sonner';

const localizer = momentLocalizer(moment);

export default function HallCalendarPage() {
  const params = useParams();
  const router = useRouter();
```

```
  const hallId = params.id as string;
  const [hall, setHall] = useState<Hall | null>(null);
  const [bookings, setBookings] = useState<Booking[]>([]);
  const [loading, setLoading] = useState(true);
  const [selectedDate, setSelectedDate] = useState(new Date());
  const [view, setView] = useState<View>('day');

  useEffect(() => {
    fetchHall();
  }, [hallId]);

  useEffect(() => {
    fetchBookings();
  }, [hallId, selectedDate]);

  const fetchHall = async () => {
    if (!hallId) return;
    try {
      const response = await api.get(`/halls/${hallId}`);
      setHall(response.data);
    } catch (error: any) {
      toast.error('Failed to fetch hall details');
    }
  };

  const fetchBookings = async () => {
    if (!hallId) return;
    try {
      // Fetch for the whole month surrounding selectedDate to be safe regardless of view
      const startOfMonth = moment(selectedDate).startOf('month').format('YYYY-MM-DD');
      const endOfMonth = moment(selectedDate).endOf('month').format('YYYY-MM-DD');

      const response = await api.get(`/availability?
start_date=${startOfMonth}&end_date=${endOfMonth}&hall_id=${hallId}`);
      setBookings(response.data || []);
    } catch (error) {
      toast.error('Failed to fetch bookings');
    } finally {
      setLoading(false);
    }
  };

  const events = (bookings || []).map((booking) => {
    const datePart = booking.event_date.split('T')[0];
    return {
      title: booking.event_title,
      start: moment(`${datePart} ${booking.start_time}`).toDate(),
      end: moment(`${datePart} ${booking.end_time}`).toDate(),
      resource: booking,
    };
  });
// ...
```